Introdução à Ciência de Dados

Módulo 1: Fundamentos de Programação em Python para transformar dados em decisões estratégicas e inteligentes

INTRODUÇÃO

O que é Ciência de Dados?

Ciência de Dados é a **área que transforma dados brutos em informação útil para tomada de decisão**

Ela está na intersecção de **três grandes pilares**:

- Estatística
- Computação
- Conhecimento de Negócio

Podemos visualizar como três áreas que se encontram e **formam um novo campo interdisciplinar**

Os Três Pilares Fundamentais

Estatística

Responsável por resumir dados, identificar padrões complexos e validar hipóteses com rigor matemático para garantir a precisão das conclusões extraídas

Computação

Permite manipular grandes volumes de dados, processar informações em larga escala e automatizar processos analíticos avançados através da programação.

Negócio

O pilar estratégico fundamental que define quais problemas resolver e qual impacto real a análise gera para a tomada de decisões corporativas mais eficientes.

Aplicações no Mundo Real

No mercado, a Ciência de Dados é aplicada em diversos setores como finanças, saúde, marketing e logística

Com ela, empresas **otimizam** operações, **preveem** tendências e melhoram a experiência do cliente, gerando vantagem competitiva **baseada em dados confiáveis e análises robustas**

Algoritmos de recomendação	Spotify e Amazon usam dados para sugerir o que você gosta
Análise de crédito	Bancos avaliam riscos de empréstimos em segundos
Previsão de demanda	Varejistas antecipam estoques para evitar perdas

INTRODUÇÃO

O Perfil do Cientista de Dados

O cientista de dados **deve possuir habilidades técnicas em estatística, programação e manipulação de dados,** além de competências analíticas e pensamento crítico

A capacidade de **comunicação** também é essencial para **traduzir insights complexos em ações estratégicas eficazes**

Pré-requisitos

- **Matemática Básica:** Domínio das quatro operações e porcentagem.
- **Lógica de Raciocínio:** Capacidade de seguir instruções sequenciais e resolver problemas simples de lógica.
- **Equipamento:** Um computador ou notebook (Windows, Mac ou Linux) com acesso à internet.
  - **Nota:** Como será utilizado o **Google Colab**, não é necessário ter um computador potente, pois o processamento ocorre na nuvem.
- **Conta Google:** Essencial para utilizar o Google Colab e salvar os arquivos no Google Drive.
- **Navegador Atualizado:** Google Chrome, Firefox ou Microsoft Edge.

Fundamentos de Programação e Python

MÓDULO 1

FUNDAMENTOS

O que é uma Linguagem de Programação?

Uma linguagem de programação **é um sistema formal de comunicação utilizado para dar instruções ao computador**

Assim como usamos o português para nos comunicar entre pessoas, **utilizamos linguagens para nos comunicarmos com máquinas,** como:

- Python
- Java
- C
- JavaScript

Em resumo, **é a forma estruturada que usamos para transformar ideias e soluções em instruções** que o computador consegue executar

FUNDAMENTOS

Python

Python é uma **linguagem de programação,** usada para desenvolvimento ágil e **análise de dados**

Sua sintaxe clara e recursos poderosos facilitam o aprendizado e a implementação de soluções complexas de forma eficiente e escalável

```python
# Trecho de exemplo em Python
def saudar_usuario(nome):
    """Esta função saúda o usuário."""
    if not nome:
        return "Olá, visitante!"
    return f"Olá, {nome}! Bem-vindo ao Python."

# Exemplo de uso
nome_pessoa = "Maria"
mensagem = saudar_usuario(nome_pessoa)
print(mensagem)
```

FUNDAMENTOS

Por que escolher Python para Ciência de Dados?

Sintaxe Intuitiva

Python foi projetado com foco em legibilidade e clareza. Isso significa que o código é mais fácil de ler, a escrita é mais intuitiva e a
a curva de aprendizado é mais suave

Ecossistema de Bibliotecas

Python possui um dos ecossistemas mais completos para Ciência de Dados, com bibliotecas que permitem trabalhar com grandes volumes de dados, criar gráficos profissionais e construir modelos preditivos

Ambiente de Desenvolvimento

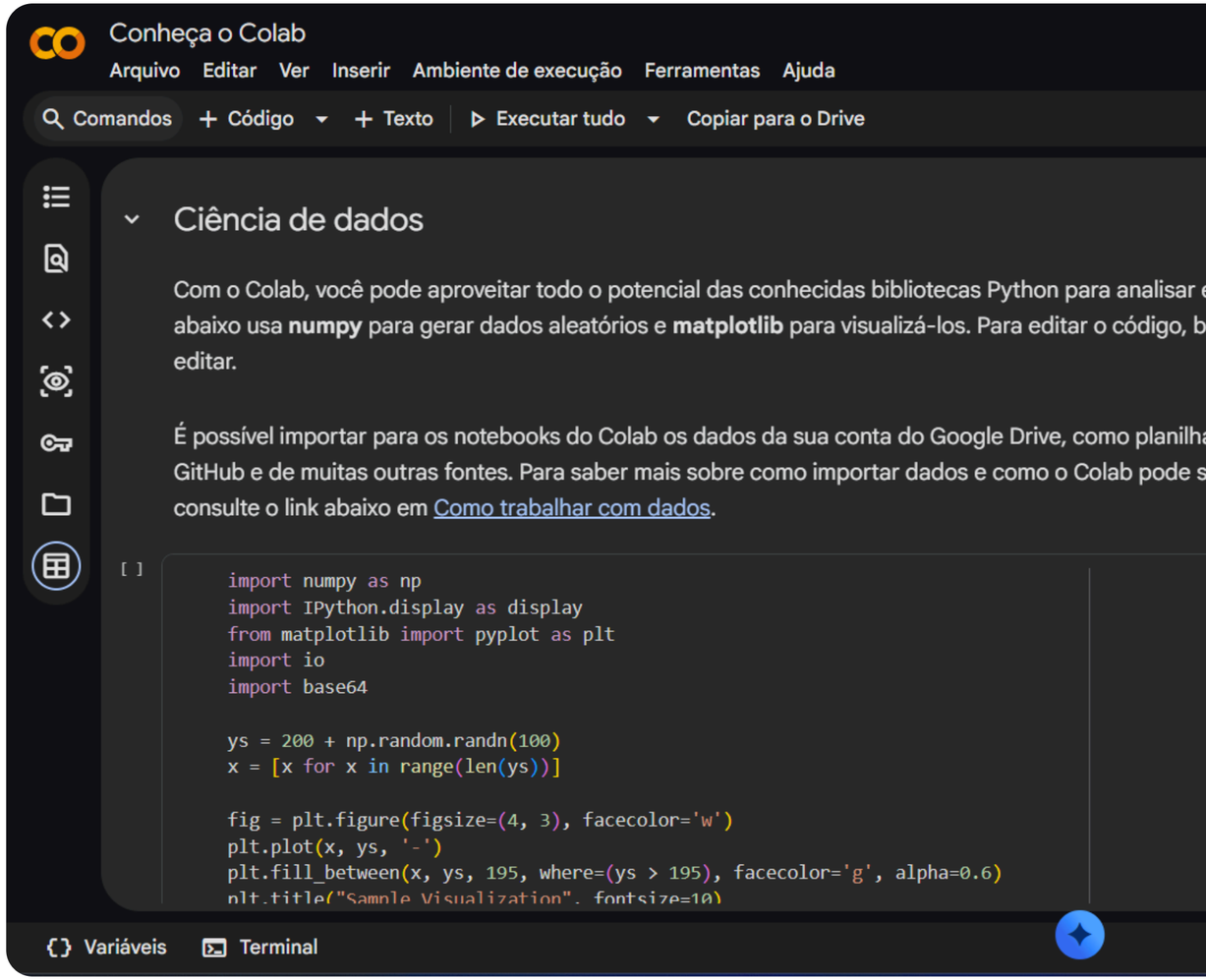
INTRODUÇÃO AO GOOGLE COLAB

PREPARAÇÃO

Google Colab

O Google Colab é um **ambiente online** que permite escrever e **executar código Python** diretamente no navegador, sem necessidade de instalação

Oferece **processamento** em nuvem, **salvamento automático** no Google Drive, e acesso a recursos avançados, facilitando o aprendizado e projetos em Ciência de Dados



GOOGLE COLAB

Notebooks

Notebooks combinam células de código e texto para executar instruções e documentar o raciocínio, permitindo o desenvolvimento interativo

Cada célula pode conter código Python executável ou explicações, tornando-os **ideais para análise, visualização e comunicação de dados**

```
[ ]
import pandas as pd

df = pd.read_csv('student-mat.csv',delimiter=';')
df
```

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob
0	GP	F	18	U	GT3	A	4	4	at_home
1	GP	F	17	U	GT3	T	1	1	at_home
2	GP	F	15	U	LE3	T	1	1	at_home

Primeiros Passos com Python

MÓDULO 1

PYTHON

Variáveis e Tipos de Dados

Uma variável é **um espaço na memória usado para armazenar informações**

Python identifica **automaticamente** o tipo do dado

1. Inteiros **(int)** - Representam números inteiros **sem** casas decimais
2. Decimais **(float)** - Representam números **com** casas decimais
3. Strings **(str)** - Representam **textos**
4. Booleanos **(bool)** - Representam valores lógicos **(verdadeiro ou falso)**

```python
# Números inteiros
idade = 25
ano = 2026

# Números decimais
preco = 19.99
altura = 1.75

# Strings
nome = "Alice"
curso = "Ciência de Dados"

# Booleanos
estudante = True
maior_de_idade = False
```

PYTHON

Entrada e Saída de Dados

Em muitos programas precisamos **interagir** com o usuário.

Isso significa que o programa deve ser capaz de:

- **Receber** informações
- **Processar** essas informações
- **Exibir** resultados

Essas operações são chamadas de **Entrada e Saída de Dados**

Entrada → A entrada de dados ocorre quando **o programa recebe informações do usuário**

Saída → A saída de dados é quando o programa **mostra informações na tela**

```python
# Utiliza-se a função print() para exibir mensagens na tela
print()

# Pode-se imprimir texto, números e variáveis
nome = "Maria"
print(nome)

# É possível imprimir mais de um item, separando-os por vírgula
idade = 22
print("Idade:", idade)

# A função input() é usada para ler dados do usuário
input()

# O valor retornado por input() é sempre uma string,
# mesmo que o usuário digite um número
nome = input("Digite seu nome: ")
print("Olá,", nome)

# Para converter a entrada do usuário para um tipo numérico
# Como inteiro, é necessário usar a função int()
idade = int(input("Digite sua idade: "))
print(idade)
```

PYTHON

Operadores Matemáticos

São utiliizados para realizar **cálculos**

Exemplo:

```
nota1 = 8
nota2 = 6
media = (nota1 + nota2) / 2
print(media)
```

Operador	Função	Exemplo
+	Soma	5 + 2 = 7
-	Subtração	5 - 2 = 3
*	Multiplicação	5 * 2 = 10
/	Divisão	5 / 2 = 2.5
//	Divisão inteira	5 // 2 = 2
%	Resto da divisão	5 % 2 = 1
**	Potência	2 ** 3 = 8

PYTHON

Condicionais

Condicionais permitem que o programa **decida o que fazer** com base em uma condição. Seguem a seguinte estrutura básica:

```
if condição:
   bloco de código
else:
   outro bloco de código
```

O programa verifica a condição:

- Se for **verdadeira** → executa o primeiro bloco
- Se for **falsa** → executa o bloco do else

```
nota = 7

if nota >= 6:
    print("Aprovado")
else:
    print("Reprovado")
```

PYTHON

Laços de Repetição

Permitem executar um bloco de código **várias vezes**

Laço for

- Usado quando **sabemos** quantas vezes queremos repetir

```python
for i in range(5):
    print(i)
```

```python
# Esse código imprime os números de 0 a 4
for i in range(5):
    print(i)

# Esse código imprime os números de 5 a 9
for i in range(5, 10):
    print(i)

# Esse código imprime os números de 0 a 9, pulando de 2 em 2
for i in range(0, 10, 2):
    print(i)

# Esse código imprime os nomes contidos na lista "nomes"
nomes = ["Alice", "Bob", "Charlie"]
for nome in nomes:
    print(nome)
```

PYTHON

Laços de Repetição

Permitem executar um bloco de código **várias vezes**

Laço while

- Usado quando a repetição **depende** de uma condição

```
contador = 0

while contador < 5:
    print(contador)
    contador += 1
```

Repete enquanto a condição for **verdadeira**

```
# Exemplo de um loop while
contador = 0

while contador < 5:
    print(contador)
    contador += 1

# Loop infinito com condição de parada
while True:
    resposta = input("Digite 'sair' para encerrar o loop: ")
    if resposta == 'sair':
        print("Loop encerrado.")
        break

# Loop infinito sem condição de parada
while True:
    print("Este loop indefinido, a menos que seja interrompido manualmente.")
```

PYTHON

Estruturas de Dados Básicas

1. Listas

Listas **armazenam múltiplos valores** em sequência

```
numeros = [10, 20, 30, 40]
```

Características:

- Ordenadas
- Permitem valores repetidos

Podem armazenar **diferentes** tipos de dados

```python
numeros = [10, 20, 30, 40]

# Acessando elementos:
print(numeros[0])

# Em Python, a contagem começa no índice 0

# Percorrendo uma lista:
for numero in numeros:
    print(numero)

# Adicionando um elemento:
numeros.append(50)

# Removendo um elemento:
numeros.remove(20)

# Alterando um elemento:
numeros[1] = 25
```

PYTHON

Estruturas de Dados Básicas

2. Dicionários

Dicionários armazenam dados no formato **chave: valor**

São usados quando queremos representar algo **estruturado**

Exemplo:

- Dados de um aluno
- Informações de um produto
- Registro de uma cidade

```python
# Criando um dicionário:
produto = {
    "nome": "Notebook",
    "preco": 3500,
    "estoque": 10
}

# Acessando valores:
print(produto["nome"])

# Adicionando um novo par chave-valor:
produto["marca"] = "Dell"

# Modificando um valor existente:
produto["preco"] = 3200

# Removendo um par chave-valor:
del produto["estoque"]
```

Boas Práticas

Programar não é apenas fazer funcionar. É fazer funcionar de maneira **clara, organizada e sustentável**

1. **Use nomes significativos**
   a. O nome da variável deve explicar o que ela representa.
2. **Mantenha padrão de escrita**
   a. Em Python, normalmente se usa letras minúsculas e palavras separadas por underline (ex: quantidade_alunos)
3. **Organize em blocos lógicos**
   a. Separe o código por etapas:
      i. Entrada de dados
      ii. Processamento
      iii. Saída de resultados
4. **Evite repetição desnecessária**
   a. Código repetido pode gerar mais erros e mais dificuldade de manutenção
   b. Sempre que possível, reutilize lógica.

```python
# Exemplo organizado:

# Entrada
nota1 = 8
nota2 = 6

# Processamento
media = (nota1 + nota2) / 2

# Saída
print(media)
```

Exercícios

- Crie um programa que imprima seu nome, idade e cidade em três linhas.
- Peça ao usuário três números a média entre eles
- Peça 3 notas ao usuário
  - Calcule média
  - Informe se foi aprovado se a média do aluno igual ou superior a 6
- Peça a idade do usuário e informe:
  - "Maior de idade" se for 18 ou mais
  - "Menor de idade" caso contrário
- Use um for para imprimir todos os números de 0 até 20.
- Use um while para imprimir apenas os números pares entre 0 e 20.
- Crie uma lista com 5 números e calcule a média dos valores da lista.
- Crie uma lista com 5 nomes e use um for para imprimir cada nome.
- Crie um programa que:
  - Peça ao usuário um número inteiro.
  - Utilize um laço for para imprimir todos os números de 1 até o número informado.
- Crie um dicionário representando um produto, contendo:
  - nome
  - preco
  - quantidade
  - Depois imprima o nome e o preço do produto

PLATAFORMA DE APRENDIZADO

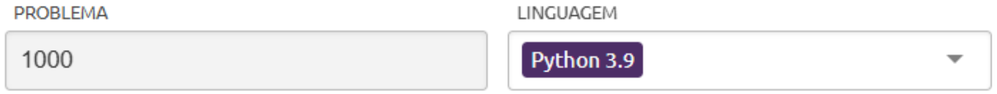
Beecrowd

O Beecrowd é uma **plataforma de aprendizado** e prática de código que oferece milhares de problemas de programação para treinar lógica e algoritmos. Para começar a utilizá-la, basta acessar o site e realizar o login diretamente com sua conta do Google

LISTA DE EXERCÍCIOS

- https://judge.beecrowd.com/pt/problems/view/1000
- https://judge.beecrowd.com/pt/problems/view/1001
- https://judge.beecrowd.com/pt/problems/view/1006
- https://judge.beecrowd.com/pt/problems/view/1020
- https://judge.beecrowd.com/pt/problems/view/1018
- https://judge.beecrowd.com/pt/problems/view/1035
- https://judge.beecrowd.com/pt/problems/view/1037
- https://judge.beecrowd.com/pt/problems/view/1080
- https://judge.beecrowd.com/pt/problems/view/1114
- https://judge.beecrowd.com/pt/problems/view/1165

OBS! SELECIONE A LINGUAGEM DE PROGRAMAÇÃO CORRETA ANTES DE ENVIAR A RESOLUÇÃO PARA VALIDAÇÃO DO PROBLEMA



Conclusão e Próximos Passos

O próximo passo é aplicar esses conceitos em conjuntos de dados reais

Até o módulo 2!